



PERFORMANCE TRACKER Football

Projet Python

Gestionnaire de Performances Football

Python · Tkinter · JSON · Matplotlib

Python

Tkinter

JSON

Matplotlib

C'est quoi ce projet ?

Le projet

Une application desktop Python permettant de gérer et suivre les performances de footballeurs.

L'utilisateur saisit les statistiques de chaque joueur : buts, passes décisives et matchs joués. Les données sont automatiquement sauvegardées et rechargées à chaque démarrage.

Fonctionnalités

Ajouter un joueur avec ses stats

Supprimer un joueur du tableau

Sauvegarde automatique (JSON)

Graphique camembert par poste

Calcul automatique du G/A

Interface dark mode Tkinter

Architecture — 3 fichiers

models.py

Logique & Données

Classes Footballeur et PerformanceManager.
Sauvegarde JSON et calculs de statistiques.

ui.py

Interface Graphique

Toute la fenêtre Tkinter :
formulaire, tableau,
boutons et graphique
Matplotlib.

main.py

Point d'entrée

Lance l'application
en 3 lignes.
Crée la fenêtre et
démarre la boucle.

main.py → importe ui.py → importe models.py · On lance uniquement python main.py

models.py — La logique

class Footballeur

`__init__()`

Crée un joueur : nom, poste, buts, passes_d, matchs. Date automatique avec date.today().

`total_individuel()`

Calcule le G/A du joueur :
return self.buts + self.passes_d

`to_dict()`

Convertit l'objet Python en dictionnaire pour pouvoir le sauvegarder en JSON.

`from_dict()`

Recrée un objet Footballeur depuis un dictionnaire lu dans le fichier JSON.

class PerformanceManager

`ajouter()`

Crée un Footballeur, l'ajoute à la liste et sauvegarde.

`supprimer(index)`

Supprime le joueur à la position donnée puis sauvegarde.

`total_buts / passes()`

Additionne les stats de tous les joueurs.

`buts_par_poste()`

Dict { poste: total_buts } pour le graphique.

`sauvegarder / charger()`

Écrit et lit le fichier footballeurs.json.

ui.py — L'interface Tkinter

`_build_header()`

Barre de titre en haut de la fenêtre

`_build_form()`

Formulaire : Nom, Buts, Passes, Matches + menu Poste

`_build_table()`

Tableau Treeview avec toutes les colonnes

`_build_footer()`

Bandeau bas affichant le total G/A global

`ajouter_footballeur()`

Valide le formulaire et appelle `manager.ajouter()`

`supprimer_footballeur()`

Récupère l'index sélectionné et supprime

`rafraichir_tableau()`

Vide et reemplit le tableau après chaque action

`afficher_graphique()`

Camembert Matplotlib des buts par poste

Concepts Python utilisés

1

Classes & `__init__`

`class Footballeur / PerformanceManager`

3

List comprehension

`[f.to_dict() for f in self.footballeurs]`

5

`json.dump` / `json.load`

Sauvegarde dans `footballeurs.json`

7

Matplotlib

`ax.pie(totaux.values(), labels=totaux.keys())`

2

`@classmethod`

`Footballeur.from_dict(data)`

4

`try / except`

`int(buts) → erreur si texte saisi`

6

Tkinter GUI

`tk.Entry`, `ttk.Treeview`, `tk.Button`

8

f-strings

`f"{self.buts} buts | {self.passes_d} passes"`

Persistence — footballeurs.json

footballeurs.json

```
[
  {
    "nom":      "Mbappé",
    "poste":    "Attaquant",
    "buts":     30,
    "passes_d": 10,
    "matches":  35,
    "date":     "2026-04-07",
    "total_ga": 40
  }
]
```

Comment ça marche ?

- 01 L'utilisateur ajoute un joueur
- 02 `to_dict()` convertit l'objet
- 03 `json.dump()` écrit le fichier
- 04 Au démarrage : `json.load()`
- 05 `from_dict()` recrée les objets

Lancer le projet

01

Créer le dossier

Créer un dossier `performance_tracker/`
Y placer : `models.py` · `ui.py` · `main.py`

02

Installer matplotlib

Ouvrir le terminal dans le dossier :
`pip install matplotlib`

03

Lancer l'application

Dans le terminal :
`python main.py`

04

C'est parti !

La fenêtre s'ouvre. Les données sont sauvegardées dans `footballeurs.json`.

Conclusion

Ce projet m'a permis de construire une vraie application Python de A à Z, en combinant plusieurs concepts fondamentaux.

Architecture propre

Séparer la logique (`models.py`) de l'interface (`ui.py`) rend le code lisible, maintenable et facile à modifier.

Concepts maîtrisés

Classes, JSON, Tkinter, Matplotlib, list comprehension, try/except — des outils essentiels en Python.

Projet fonctionnel

L'application tourne, sauvegarde les données et affiche un graphique. C'est un vrai projet utilisable.